

INFLUENCELIFT AI

System Design Portfolio Defense Guide

A 42-page interview manual for explaining the architecture, trade-offs, reliability, and evolution of InfluenceLift AI.

Portfolio Edition 1.0

Primary author	Mohit Bhatnagar
Collaboration credit	Aarushi Rai - marketing analytics framing and portfolio positioning
Live application	https://influencelift-ai.streamlit.app
Source repository	https://github.com/mohit231007/influencelift-ai

Purpose: help the project owner explain, defend, extend, and market the system in technical and business interviews.

How to use this guide

Use modules 1-10 for a first-round explanation, 11-30 for deep technical follow-ups, and 31-40 for senior-level trade-off and defense practice.

This is a practical interview and portfolio manual. Each numbered module is intentionally self-contained so you can revise one topic, rehearse a two-minute answer, and then practise a deeper follow-up.

Module	Topic
01	Problem framing and success criteria
02	Functional and non-functional requirements
03	System context and actors
04	Container architecture
05	End-to-end data flow
06	Messy-data parsing strategy
07	Repair policy and auditability
08	Schema validation and contracts
09	Feature engineering philosophy
10	Model selection and baseline discipline
11	Cross-validation design
12	Metric selection
13	Prediction intervals and uncertainty
14	Extrapolation detection
15	Model bundle and versioning
16	FastAPI interface design
17	Streamlit product design
18	Scenario simulation
19	Batch inference architecture
20	Testing strategy
21	CI/CD pipeline
22	Docker and runtime portability
23	Configuration management
24	Logging and observability
25	Model monitoring and drift
26	Security and privacy
27	Responsible AI and causal boundaries
28	Performance and latency
29	Scalability roadmap
30	Data storage design
31	Failure modes and graceful degradation
32	Cost architecture
33	Repository and package architecture
34	Deployment strategy
35	Business decision integration
36	Technical debt register
37	Roadmap and evolution
38	Whiteboard defense sequence
39	Two-minute project defense
40	Deep-dive defense checklist

01. Problem framing and success criteria

Objective: Translate a broad campaign-forecasting request into measurable product and ML outcomes.

Core explanation

- Separate the business decision from the modelling task: estimate sales before budget commitment.
- Define user groups: marketing analyst, campaign manager, data scientist, API consumer, and recruiter evaluating the portfolio.
- Use measurable acceptance criteria: valid messy inputs, deterministic cleaning, reproducible training, stable cross-validation, non-negative output, and downloadable batch results.

Portfolio defense answer

I started by defining the decision the system supports, not by choosing an algorithm. The system must convert unreliable campaign records into an auditable forecast and a scenario comparison that a marketing user can understand. Model accuracy matters, but reliability, transparency, and deployability are equal success criteria.

Evidence in InfluenceLift AI

- README states the two-part problem: make messy campaign data reliable, then forecast sales.
- The public app supports single prediction, batch prediction, scenario simulation, and model information.
- The model card distinguishes decision support from causal proof.

Trade-off to acknowledge

A narrow feature set limits ceiling accuracy, but it creates a clear baseline and forces the portfolio to demonstrate engineering quality rather than hide behind model complexity.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Deliver a 90-second answer covering business problem, target user, prediction target, and acceptance criteria.

02. Functional and non-functional requirements

Objective: Defend what the system must do and the quality attributes it must preserve.

Core explanation

- Functional requirements include parsing, auditing, training, predicting, batch upload, scenario comparison, API access, and result download.
- Non-functional requirements include reproducibility, low operational cost, understandable error handling, testability, portability, and safe handling of public data.
- Requirements are prioritised using must-have, should-have, and future categories.

Portfolio defense answer

The must-have flow is input validation through forecast delivery. The non-functional design is intentionally stronger than a notebook: transformations live inside the pipeline, tests run on multiple Python versions, Docker captures runtime dependencies, and the live demo uses synthetic data to respect redistribution constraints.

Evidence in InfluenceLift AI

- GitHub Actions tests Python 3.10-3.12.
- Dockerfile and docker-compose support portable execution.
- Data policy prevents accidental publication of original challenge files.

Trade-off to acknowledge

The current app favours simplicity over authentication and persistent storage. That is appropriate for a public portfolio demo but not for a client system.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Classify ten requirements into functional, reliability, security, maintainability, and usability groups.

03. System context and actors

Objective: Explain the system boundary and external dependencies.

Core explanation

- Primary actor: campaign decision-maker using Streamlit.
- Secondary actor: developer or downstream service using FastAPI.
- External systems: GitHub, Streamlit Community Cloud, optional container runtime, and future campaign data sources.
- The model artefact and synthetic sample data are internal system assets.

Portfolio defense answer

The context boundary is deliberately small. InfluenceLift AI accepts a campaign record or CSV, performs all cleaning and inference inside one deployable application, and returns predictions plus audit information. No third-party API is required for inference, which reduces latency, cost, and secret-management risk.

Evidence in InfluenceLift AI

- No API key is needed for the hosted demo.
- The Streamlit app can generate or load the demo model locally.
- FastAPI exposes health, model-info, predict, and batch endpoints.

Trade-off to acknowledge

A self-contained system is easier to demo but less representative of an enterprise data platform with a feature store, registry, and warehouse.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Draw a context diagram in under three minutes and state what is outside the boundary.

04. Container architecture

Objective: Describe the major deployable/runtime units and their responsibilities.

Core explanation

- Streamlit is the human-facing decision interface.
- FastAPI is the programmatic contract.
- The influencelift Python package owns cleaning, training, inference, and simulation logic.
- A serialized model bundle stores estimator, metrics, residual band, and training ranges.
- GitHub Actions provides automated quality gates.

Portfolio defense answer

I separated interface layers from reusable domain logic. Streamlit and FastAPI are thin adapters; they call the same package functions. This prevents divergent business rules and keeps the source of truth in the Python package.

Evidence in InfluenceLift AI

- The package is installed from pyproject.toml.
- Both interfaces call shared inference functions.
- The Docker image installs the package before copying interface code.

Trade-off to acknowledge

The repository is a modular monolith, not microservices. That choice avoids network complexity for a small workload while preserving clear boundaries for future extraction.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Explain why modular monolith is the right architecture today and identify the first component you would extract.

05. End-to-end data flow

Objective: Trace one request from raw input to response.

Core explanation

- Validate required columns or request fields.
- Parse symbols, suffixes, commas, and missing tokens.
- Apply transparent repair rules and produce audit flags.
- Impute missing numeric values, standardise features, and run Ridge inference.
- Clamp negative predictions, add empirical bounds, attach corrections and extrapolation warnings.
- Present or serialize the result.

Portfolio defense answer

The key design decision is that the exact cleaning transformer used during training is also part of inference. This removes training-serving skew. Audit logic runs alongside the model path so the user sees what changed, rather than receiving an unexplained number.

Evidence in InfluenceLift AI

- CampaignCleaner implements scikit-learn TransformerMixin.
- Prediction output includes corrections, warning fields, and model version.
- Batch results preserve identifiers and add prediction columns.

Trade-off to acknowledge

The audit pass duplicates some parsing work. At this scale the clarity is worth the minor overhead; a high-volume service could return audit metadata directly from a single transformation pass.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Walk through the `125K`, `3.7%`, `£5,500`, `8.2` example end to end.

06. Messy-data parsing strategy

Objective: Defend how human-formatted marketing data becomes numeric data.

Core explanation

- Normalize empty tokens such as blank, null, N/A, and dash.
- Remove currency, percentage, whitespace, and separators.
- Support K, M, and B suffixes.
- Return missing rather than throwing on malformed values, allowing controlled imputation and audit.
- Keep parsing deterministic and unit-testable.

Portfolio defense answer

I treated parsing as a domain capability rather than ad-hoc notebook cleaning. The parser is intentionally strict: it accepts known human formats and turns unknown text into missing data. That behaviour is safer than guessing and easier to test.

Evidence in InfluenceLift AI

- `parse_human_number` uses a full-match regular expression.
- Currency symbols include GBP, USD, EUR, and INR.
- Unit tests cover suffixes and malformed values.

Trade-off to acknowledge

Returning NaN can hide severe schema problems if audit output is ignored. Production policy should set rejection thresholds for excessive invalid values.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Design three extra parser tests for parentheses negatives, decimal commas, and whitespace-heavy inputs.

07. Repair policy and auditability

Objective: Explain why heuristic repairs are configurable and observable.

Core explanation

- CleaningPolicy centralises thresholds and toggles.
- Small follower values may be interpreted as thousands for this dataset.
- Inflated followers and spend are repaired by configured divisors.
- Negative engagement/spend can be treated as sign errors.
- Every correction receives a row-level flag and aggregate count.

Portfolio defense answer

The original dataset contained systematic scale and sign corruption. I encoded those findings as an explicit policy object rather than burying magic numbers in a notebook. A new organisation can disable or replace the heuristics after profiling its own data.

Evidence in InfluenceLift AI

- CleaningPolicy exposes thresholds and divisors.
- DataQualityReport distinguishes corrections from invalid values.
- The UI converts internal codes into human-readable explanations.

Trade-off to acknowledge

Dataset-specific repairs can be dangerous when reused blindly. The model card therefore calls out the need to validate policy thresholds before deployment.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Answer: why is converting negative spend to absolute value defensible here, and when would you reject it instead?

08. Schema validation and contracts

Objective: Show how the system fails early and predictably.

Core explanation

- Raw batch input requires four named campaign feature columns.
- API requests use typed Pydantic schemas.
- Validation errors should identify missing fields and preserve HTTP semantics.
- Model output has a stable response schema including uncertainty and metadata.

Portfolio defense answer

I use separate contracts for tabular and API inputs but converge them into the same internal feature schema. Missing required columns raise a clear error before any model call; malformed API payloads produce a 422 response automatically.

Evidence in InfluenceLift AI

- ``validate_required_columns`` lists the exact missing columns.
- FastAPI exposes OpenAPI documentation.
- Response models include bounds, status, corrections, warnings, and version.

Trade-off to acknowledge

Column-name coupling is simple but brittle. Enterprise ingestion would add mapping configuration, schema versioning, and a quarantine path.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Propose a backward-compatible v2 request that supports currency code and campaign ID.

09. Feature engineering philosophy

Objective: Defend why the production model uses a small, interpretable feature set.

Core explanation

- Use the four supplied drivers as the baseline.
- Standardisation makes Ridge regularisation comparable across mixed scales.
- Median imputation keeps training and inference robust.
- Log or interaction features should be retained only if cross-validation improves and business meaning remains clear.

Portfolio defense answer

I tested richer alternatives but selected the stable regularised baseline. The portfolio demonstrates disciplined model selection: complexity is earned through validation, not added for appearance.

Evidence in InfluenceLift AI

- Model comparison documented Ridge versus linear regression and XGBoost.
- The pipeline standardises numeric features.
- Scenario output maps directly to understandable business inputs.

Trade-off to acknowledge

The linear form can miss saturation, thresholds, and interactions. A future challenger could use monotonic gradient boosting or GAMs while preserving explainability.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Describe three plausible interactions and how you would validate them without leakage.

10. Model selection and baseline discipline

Objective: Explain the hierarchy of models and the selection rule.

Core explanation

- Start with a naive mean baseline.
- Compare ordinary linear regression, regularised linear models, and nonlinear ensembles.
- Use identical cross-validation folds and metrics.
- Select based on generalisation, stability, interpretability, latency, and operational burden.

Portfolio defense answer

Tuned Ridge and linear regression performed similarly, while tuned XGBoost was slightly worse. Ridge was selected because it preserved accuracy while adding coefficient stability and lower operational complexity.

Evidence in InfluenceLift AI

- Five-fold RMSE for Ridge: about 2,000 units.
- R^2 about 0.493 on original case-study data.
- Fold RMSE standard deviation about 44.57 units.

Trade-off to acknowledge

The accuracy gap was small, so model choice is sensitive to data revisions. A challenger framework should be rerun when more campaigns arrive.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Defend the sentence: “I did not choose XGBoost because it was more sophisticated.”

11. Cross-validation design

Objective: Defend the validation method and identify when it should change.

Core explanation

- Use out-of-fold predictions to estimate generalisation.
- Keep cleaning, imputation, scaling, and model fitting inside each fold.
- Report mean and dispersion, not only one score.
- Use time-based validation if campaign chronology creates leakage or distribution shift.

Portfolio defense answer

The case study used five-fold cross-validation because the available sample was treated as exchangeable. The pipeline prevents preprocessing leakage. If timestamp order reflects changing platforms or seasonality, I would switch to rolling or blocked time splits.

Evidence in InfluenceLift AI

- Cross-validation metrics are stored with the model bundle.
- The model card reports fold stability.
- The demo training script can reproduce metrics.

Trade-off to acknowledge

Random folds may overstate performance when similar creators or campaign periods appear in both train and validation sets.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Compare KFold, GroupKFold by influencer, and TimeSeriesSplit for this use case.

12. Metric selection

Objective: Connect RMSE, MAE, and R^2 to stakeholder decisions.

Core explanation

- RMSE penalises large campaign misses.
- MAE communicates a typical absolute unit error.
- R^2 describes variance explained relative to the mean baseline.
- Prediction interval coverage is needed to communicate uncertainty.
- Segment metrics can expose unfair or unstable performance.

Portfolio defense answer

I use a metric portfolio. RMSE is the optimisation-oriented headline because large misses can damage inventory or budget decisions; MAE is easier to interpret; R^2 gives relative context. None of them should be presented without the target scale and business consequences.

Evidence in InfluenceLift AI

- Original case-study metrics appear in README and model card.
- Demo metrics are labelled separately because synthetic data differs.
- The API returns a residual-based interval.

Trade-off to acknowledge

R^2 can be misleading when the target distribution or baseline changes. Business evaluation should also use margin-weighted error or decision regret.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Explain how a 2,000-unit RMSE could be acceptable for one product and unacceptable for another.

13. Prediction intervals and uncertainty

Objective: Explain the empirical interval and its limitations.

Core explanation

- Calculate an absolute residual quantile from out-of-fold predictions.
- Apply the residual band around point forecasts.
- Clamp the lower bound at zero.
- Label the range as empirical rather than guaranteed conditional coverage.
- Monitor realised coverage when outcomes arrive.

Portfolio defense answer

The interval prevents false precision. It is a simple residual band suitable for a baseline demo. I explicitly avoid calling it a perfect 95% confidence interval because error variance may depend on campaign scale.

Evidence in InfluenceLift AI

- Model bundle stores `residual_quantile_95`.
- Streamlit shows lower and upper interval.
- Model card states that conditional coverage is not guaranteed.

Trade-off to acknowledge

A constant-width band is less accurate under heteroscedasticity. Conformal prediction, quantile regression, or scale-dependent residual models are stronger future options.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Explain the difference between a confidence interval for a mean and a prediction interval for a future campaign.

14. Extrapolation detection

Objective: Show how the system warns when a request is unlike training data.

Core explanation

- Store min/max ranges for each trained feature.
- Compare cleaned request values to those ranges.
- Attach warnings rather than silently extrapolating.
- Use warning frequency as a monitoring signal.
- Future versions can use multivariate distance rather than independent ranges.

Portfolio defense answer

The model still returns a result because this is decision support, but the response tells the user when a feature lies outside observed training ranges. That creates a safer conversation than either silent extrapolation or a hard failure for every boundary case.

Evidence in InfluenceLift AI

- Training ranges appear in the Model information tab.
- Prediction response includes `extrapolation\_warnings`.
- Batch output contains warning columns.

Trade-off to acknowledge

Min/max checks miss unusual combinations inside marginal ranges. Mahalanobis distance, isolation forests, or density estimation could improve coverage.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Give an example where every feature is within range but the combination is still out of distribution.

15. Model bundle and versioning

Objective: Explain what is serialized and how the system stays reproducible.

Core explanation

- Bundle the fitted pipeline, model version, alpha, metrics, residual band, and training ranges.
- Load one immutable bundle for inference.
- Expose model version in UI and API responses.
- Keep generated model artefacts outside Git by default.
- Use registry-backed promotion for enterprise environments.

Portfolio defense answer

I serialize more than the estimator because a production prediction needs preprocessing, uncertainty, range checks, and metadata. Version exposure makes screenshots and API results traceable to the exact demo model.

Evidence in InfluenceLift AI

- The live app displays demo-1.0.0.
- Environment variables allow a model path and version override.
- Artifacts directory is ignored except documentation.

Trade-off to acknowledge

Joblib is convenient but not a secure interchange format; never load untrusted artefacts. ONNX or a controlled registry can reduce risk.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Design a semantic versioning rule for data changes, feature changes, and estimator-only changes.

16. FastAPI interface design

Objective: Defend the API resources, schemas, and response semantics.

Core explanation

- `GET /health` supports liveness checks.
- `GET /model-info` exposes non-sensitive metadata.
- `POST /predict` handles one campaign.
- `POST /predict-batch` supports multiple records.
- Pydantic documents and validates the contract.

Portfolio defense answer

The API is intentionally small and stateless. Each endpoint has one responsibility, and the response includes both the forecast and the information needed to judge it. OpenAPI documentation makes the service testable without a custom client.

Evidence in InfluenceLift AI

- Swagger UI was manually validated locally.
- The same messy strings used in Streamlit work in the API.
- A 200 response matched the UI prediction during QA.

Trade-off to acknowledge

The current batch request is JSON-based; very large batches should use object storage, asynchronous jobs, and status polling.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Design an idempotent asynchronous batch API for one million campaign rows.

17. Streamlit product design

Objective: Explain why the UI is structured around decisions rather than model internals.

Core explanation

- Campaign predictor supports fast individual what-if checks.
- Batch predictions support operational analyst workflow.
- Scenario simulator compares baseline and proposed campaigns.
- Model information separates original benchmark from synthetic demo model.
- Human-readable correction messages replace internal flags.

Portfolio defense answer

The interface is designed for a non-technical reviewer to understand the problem in under a minute. The model is visible, but the workflow starts with campaign decisions and data quality. That is more valuable than displaying dozens of diagnostic charts.

Evidence in InfluenceLift AI

- Public demo runs without secrets.
- Single prediction and download flows were manually verified.
- UI clearly labels predictive associations as non-causal.

Trade-off to acknowledge

Streamlit is excellent for prototypes but offers less control than a dedicated frontend. A scaled product could move to React while preserving the same API.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Critique the current UI from the perspective of a busy marketing manager.

18. Scenario simulation

Objective: Defend the baseline-versus-proposed comparison and its guardrails.

Core explanation

- Run both scenarios through the identical model bundle.
- Calculate expected uplift as forecast difference.
- Calculate incremental cost per predicted sale when spend increases.
- Show association disclaimer.
- Warn when either scenario extrapolates.

Portfolio defense answer

The simulator turns a regression model into a decision-support experience. It does not claim that manually changing spend causes the exact uplift; it shows how the fitted relationship changes the prediction while making uncertainty and causal limitations explicit.

Evidence in InfluenceLift AI

- Default scenario produced a visible uplift in local QA.
- Simulation code is separated from the UI.
- Inputs map directly to the four model features.

Trade-off to acknowledge

Cost per predicted sale ignores margin, cannibalisation, baseline demand, and creator fees outside ad spend. Real optimisation needs a richer objective.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Extend the simulator to optimise profit under a fixed budget and inventory cap.

19. Batch inference architecture

Objective: Explain how CSV processing remains consistent and auditable.

Core explanation

- Preview uploaded data.
- Audit the entire batch before prediction.
- Display summary counts and optional technical detail.
- Preserve IDs and append prediction, bounds, corrections, and warnings.
- Enable CSV download.

Portfolio defense answer

Batch inference uses the same model path as single prediction, so there is no separate cleaning implementation. The audit summary helps an analyst decide whether to trust or repair the file before operational use.

Evidence in InfluenceLift AI

- Synthetic sample contains deliberate missing and malformed values.
- Downloaded file was validated for 30 rows and ordered intervals.
- Errors are caught and displayed rather than crashing the app.

Trade-off to acknowledge

In-memory CSV processing is not suitable for very large files. Chunked processing or asynchronous workers would be required.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Describe memory-safe processing for a 10 GB campaign file.

20. Testing strategy

Objective: Defend the test pyramid and identify missing tests.

Core explanation

- Unit tests cover parser and cleaning rules.
- Integration tests cover training pipeline and API.
- Smoke tests start actual Streamlit and FastAPI processes.
- Manual acceptance tests validate interactive flows and downloads.
- Multi-version CI protects supported Python versions.

Portfolio defense answer

I did not treat a green notebook as proof of quality. The project has unit, integration, runtime smoke, and manual product checks. The Windows QA script also validates the exact workflow a recruiter or contributor may run.

Evidence in InfluenceLift AI

- Seven automated tests passed locally.
- CI runs Python 3.10, 3.11, and 3.12.
- Deployment smoke workflow checks both service health endpoints.

Trade-off to acknowledge

Coverage is still limited. Property-based parser tests, load tests, visual regression, and failure-injection tests would strengthen production readiness.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Propose five high-value tests that are not currently present.

21. CI/CD pipeline

Objective: Explain the automated quality gate and deployment relationship.

Core explanation

- Checkout and install the package.
- Run Ruff, pytest, and compile checks.
- Run a separate deployment smoke workflow.
- Block merge until checks pass.
- Streamlit Community Cloud redeploys from main.

Portfolio defense answer

Every material change is isolated in a branch and merged only after automated checks. The hosted app tracks main, so the repository remains the deployment source of truth.

Evidence in InfluenceLift AI

- PRs #1-#4 were merged after green checks.
- Deployment smoke tests start real processes.
- CI matrix covers three Python versions.

Trade-off to acknowledge

The workflow does not yet build and scan a published container image. Adding image signing and environment promotion would be required for enterprise delivery.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Design dev, staging, and production promotion without retraining the model in each environment.

22. Docker and runtime portability

Objective: Defend container choices and note production hardening.

Core explanation

- Use a slim Python base image.
- Install package dependencies before interface code to improve cache behaviour.
- Expose Streamlit and FastAPI ports.
- Use docker-compose for local multi-service launch.
- Avoid embedding secrets and original data.

Portfolio defense answer

Docker gives a consistent demonstration path and makes the runtime explicit. The repository still supports a simple virtual environment, so users are not forced into containers.

Evidence in InfluenceLift AI

- Dockerfile installs from pyproject.toml.
- docker-compose launches both UI and API.
- The image copies synthetic sample data only.

Trade-off to acknowledge

The current container runs as root and lacks a healthcheck in the Dockerfile. Production hardening should use a non-root user, pinned hashes, health checks, and vulnerability scans.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Explain how you would reduce image size and supply-chain risk.

23. Configuration management

Objective: Explain the boundary between code, configuration, and secrets.

Core explanation

- Environment variables control model path, version, and log level.
- Cleaning thresholds are represented as a typed policy.
- Streamlit visual configuration lives in ``.streamlit/config.toml``.
- No secrets are required for the public demo.
- Future credentials belong in platform secret stores.

Portfolio defense answer

I keep stable defaults in code and deployment-specific values in environment variables. The public demo has no external dependency, so it avoids unnecessary secret management.

Evidence in InfluenceLift AI

- ``.env.example`` documents variables.
- Git ignores ``.env``.
- Streamlit Community Cloud deployment used an empty secrets field.

Trade-off to acknowledge

Too many environment variables can become unvalidated global state. A typed settings object and environment-specific configuration files would improve larger deployments.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Design configuration validation that fails fast on startup.

24. Logging and observability

Objective: Describe what should be logged, measured, and protected.

Core explanation

- Log request ID, model version, latency, status, and aggregate correction counts.
- Never log raw sensitive campaign notes or credentials.
- Measure prediction volume, error rate, latency percentiles, and out-of-range frequency.
- Track data drift and model error when actual sales arrive.
- Add dashboards and alerts with actionable thresholds.

Portfolio defense answer

The demo exposes health and model information, but production observability would connect operational and model metrics. The most important domain signal is not CPU usage; it is rising correction or extrapolation frequency.

Evidence in InfluenceLift AI

- Data-quality report already calculates correction counts.
- Model bundle includes version and metrics.
- Health endpoint enables platform monitoring.

Trade-off to acknowledge

Observability can create privacy risk and storage cost. Logging must be minimised, structured, sampled, and retained according to policy.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Propose a dashboard with six operational and six model-quality metrics.

25. Model monitoring and drift

Objective: Explain the monitoring loop after deployment.

Core explanation

- Monitor feature distributions and missingness.
- Track correction, invalid, and extrapolation rates.
- Join predictions to actual sales when available.
- Measure RMSE, MAE, bias, and interval coverage over time.
- Slice by product, creator tier, geography, and campaign period.
- Retrain only after diagnosis and validation.

Portfolio defense answer

A live model is not finished at deployment. InfluenceLift AI already emits the metadata needed for a basic monitoring layer. The next step is to persist predictions and outcomes, then compare performance against training and business thresholds.

Evidence in InfluenceLift AI

- Model card lists monitoring recommendations.
- Prediction output includes model version and warnings.
- Synthetic demo makes no claim of real-time production monitoring.

Trade-off to acknowledge

Drift does not always mean performance degradation, and stable inputs do not guarantee stable attribution. Retraining should not be automatic without outcome evidence.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Explain concept drift, covariate drift, label drift, and data-quality drift using this project.

26. Security and privacy

Objective: Defend the security posture of a public ML demo.

Core explanation

- Use synthetic data and publish no client records.
- Keep secrets out of Git and environment examples.
- Validate inputs and limit accepted formats.
- Do not load untrusted model artefacts.
- Apply dependency scanning and least privilege in production.
- Rate-limit and authenticate if exposing sensitive predictions.

Portfolio defense answer

The strongest security decision was reducing the attack surface: no database, no API keys, no uploaded data persistence, and no original challenge dataset in the repository. Production security would add authentication, limits, audit logs, and hardened containers.

Evidence in InfluenceLift AI

- Security policy exists.
- Data policy is explicit in README.
- API only provides inference operations.

Trade-off to acknowledge

Public file upload can still be abused for memory exhaustion. Input size limits and content validation should be added.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Threat-model the upload endpoint using STRIDE.

27. Responsible AI and causal boundaries

Objective: Explain ethical and analytical guardrails.

Core explanation

- Predictions are associations from observational data.
- Do not automate creator employment or contracting decisions.
- Monitor performance by segments where relevant.
- Avoid unnecessary personal attributes.
- Communicate uncertainty and human review.
- Document intended and out-of-scope use.

Portfolio defense answer

The model supports campaign planning; it does not rank human worth or prove causal lift. I deliberately exclude personal attributes and state that creator-contract decisions require broader evidence and human judgment.

Evidence in InfluenceLift AI

- Model card contains intended and out-of-scope use.
- UI displays non-causal disclaimer.
- Prediction intervals reduce false precision.

Trade-off to acknowledge

Even apparently neutral features such as follower count can proxy for structural advantages. Fairness evaluation depends on the actual decision and affected groups.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Answer how you would prevent this system from becoming an automatic creator rejection tool.

28. Performance and latency

Objective: Estimate bottlenecks and defend optimisation priorities.

Core explanation

- Ridge inference is negligible compared with app startup and file parsing.
- Cache the model bundle once per process.
- Vectorise batch parsing and prediction.
- Set upload limits and use chunking for large files.
- Measure before optimising.

Portfolio defense answer

The current model is intentionally lightweight. The user experience is dominated by cold start and dependency loading, not matrix multiplication. I cache the model resource and keep inference local to avoid network latency.

Evidence in InfluenceLift AI

- Streamlit uses resource caching.
- No external inference API is called.
- Batch prediction uses pandas and the pipeline.

Trade-off to acknowledge

The row-level audit iterates through records and could become a bottleneck at high volume. Vectorised flag generation or compiled parsing would be the first optimisation.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Estimate latency for one row, 10,000 rows, and one million rows and identify architecture changes.

29. Scalability roadmap

Objective: Explain how the design evolves without premature microservices.

Core explanation

- Phase 1: modular monolith and public demo.
- Phase 2: persistent object storage, job queue, model registry, and monitoring.
- Phase 3: asynchronous batch workers and autoscaled API.
- Phase 4: multi-tenant security, feature store, and governed retraining.
- Extract components only when load or ownership justifies it.

Portfolio defense answer

I would scale the workload boundary, not copy every function into a service. Large batch jobs would be the first separate worker because their resource profile differs from interactive prediction.

Evidence in InfluenceLift AI

- Package boundaries already isolate cleaning and inference.
- FastAPI provides a future service interface.
- Docker creates a portable deployment unit.

Trade-off to acknowledge

Distributed architecture increases operational failure modes, cost, and debugging time. The portfolio intentionally avoids claiming enterprise scale it has not earned.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Design a 10x and 1,000x scale plan and state the trigger for each change.

30. Data storage design

Objective: Explain what should be persisted in a production version.

Core explanation

- Store raw input reference, cleaned feature snapshot, prediction, bounds, warnings, model version, and timestamp.
- Keep immutable raw data separate from derived records.
- Use object storage for files and relational storage for metadata.
- Apply retention and access policies.
- Avoid storing unnecessary notes or personal information.

Portfolio defense answer

The demo is stateless by design. A production system needs traceability, so I would persist a compact prediction ledger and keep uploaded files in controlled object storage. This supports audit, monitoring, and reproducibility.

Evidence in InfluenceLift AI

- Current batch download gives a portable result ledger.
- Model version is already returned.
- No uploaded file is persisted by the demo.

Trade-off to acknowledge

Persistence introduces privacy, deletion, and access-control obligations. “Store everything” is not an acceptable default.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Sketch a relational schema for campaigns, prediction runs, model versions, and outcomes.

31. Failure modes and graceful degradation

Objective: Show how the system behaves when components fail.

Core explanation

- Invalid schema: fail with explicit missing fields.
- Malformed values: flag and impute where policy permits.
- Missing model artefact: generate/load deterministic demo bundle or fail startup clearly.
- Large upload: reject before memory exhaustion.
- Inference exception: return safe error without stack trace.
- Hosted cold start: show progress rather than duplicate work.

Portfolio defense answer

A good ML product is defined by failure behaviour as much as its happy path. I separate repairable data issues from invalid records and keep model loading deterministic. Production would add explicit timeouts, size limits, retries for external stores, and a dead-letter path for batch jobs.

Evidence in InfluenceLift AI

- Streamlit catches value/type errors for batch uploads.
- FastAPI validation returns structured errors.
- QA script surfaced and fixed a stale Windows Python launcher issue.

Trade-off to acknowledge

Automatic imputation can allow low-quality batches to proceed. A production policy should stop runs when invalid-rate thresholds are exceeded.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Create a failure-mode-and-effects table with severity, detection, and mitigation.

32. Cost architecture

Objective: Explain why the current solution is inexpensive and where costs appear later.

Core explanation

- Ridge requires minimal CPU and memory.
- Streamlit Community Cloud hosts the public app at no direct cost.
- GitHub Actions usage is small for this repository.
- No paid inference API or database is required.
- Future costs come from storage, monitoring, batch compute, authentication, and support.

Portfolio defense answer

The architecture matches the portfolio objective: a reliable live demo with near-zero operating cost. I would not introduce a paid vector database, GPU, or managed feature store when the workload does not require them.

Evidence in InfluenceLift AI

- The public URL runs without secrets.
- Synthetic model loads locally.
- Docker can run on commodity infrastructure.

Trade-off to acknowledge

Free tiers can sleep, change limits, or lack service guarantees. A business deployment needs a costed SLA-backed platform.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Prepare a monthly cost estimate for demo, small team, and enterprise scenarios.

33. Repository and package architecture

Objective: Explain why the repository is structured as a product.

Core explanation

- Separate app, API, package, scripts, tests, data samples, docs, and reports.
- Use pyproject.toml as the packaging and dependency source.
- Keep generated artefacts and raw data out of version control.
- Provide contribution, security, model-card, and citation files.
- Use scripts for reproducible command-line workflows.

Portfolio defense answer

The repository communicates engineering maturity before a reviewer runs the code. A notebook remains available for analysis, but reusable logic lives in importable modules and tested interfaces.

Evidence in InfluenceLift AI

- README includes architecture and quick-start paths.
- MIT licence and contribution guidance are included.
- CITATION.cff supports academic-style attribution.

Trade-off to acknowledge

A single repository is convenient now; multiple teams may eventually need separate package and deployment repositories.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Explain the role of every top-level directory without looking at the README.

34. Deployment strategy

Objective: Defend the chosen public hosting and an enterprise alternative.

Core explanation

- Streamlit Community Cloud gives a permanent recruiter-accessible URL.
- Main branch is the deployment source.
- No secrets or external services simplify deployment.
- Enterprise alternative: container platform plus managed API, object storage, database, and monitoring.
- Use environment promotion and immutable images.

Portfolio defense answer

I selected Community Cloud because it optimises the actual requirement: recruiters can click and use the product for free. I would not present it as an enterprise SLA platform. The Docker and API layers demonstrate a credible migration path.

Evidence in InfluenceLift AI

- Live application is publicly verified.
- README badge links directly to the app.
- Deployment documentation and smoke workflow exist.

Trade-off to acknowledge

Community Cloud can sleep and has resource constraints. A client-facing service would need a supported runtime and operational ownership.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Compare Streamlit Community Cloud, Render, Azure App Service, and Kubernetes for this workload.

35. Business decision integration

Objective: Show how the forecast fits a broader campaign process.

Core explanation

- Combine predicted units with margin, inventory, creator fit, and campaign objective.
- Use scenario comparison before approval.
- Record the final human decision and rationale.
- Compare predicted versus actual outcomes.
- Use learnings to update policy and features.

Portfolio defense answer

InfluenceLift AI is one input to a campaign brief, not an automatic budget allocator. A decision dashboard should combine forecast, uncertainty, quality status, unit economics, and qualitative brand-fit review.

Evidence in InfluenceLift AI

- Model card lists additional decision factors.
- Scenario simulator exposes cost per predicted sale.
- Output is downloadable for approval workflows.

Trade-off to acknowledge

Optimising predicted units alone can favour low-margin or brand-inappropriate campaigns. The objective must reflect business value.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Create a one-page campaign approval template using the model output.

36. Technical debt register

Objective: Identify known compromises and prioritise them.

Core explanation

- Row-wise audit performance.
- Constant-width residual interval.
- Limited automated test coverage.
- No persistent prediction ledger.
- No authentication or rate limiting.
- No formal model registry or drift dashboard.
- Synthetic demo differs from original case-study model.

Portfolio defense answer

I keep a visible debt register rather than claiming production completeness. The highest priorities depend on deployment context: input limits and authentication for exposure, then persistent monitoring for real decisions.

Evidence in InfluenceLift AI

- Roadmap lists explainability, MLflow, drift, and optimisation.
- Model information explicitly labels demo metrics.
- Security and model-card documents describe limitations.

Trade-off to acknowledge

Adding features without paying down observability and data-contract debt can make the system less trustworthy.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Rank the debt items for a recruiter demo, an internal pilot, and a regulated client.

37. Roadmap and evolution

Objective: Explain a credible next release without over-engineering.

Core explanation

- v1.1: coefficient/local contribution explanation, richer quality summary, contract tests.
- v1.2: prediction ledger, outcome upload, drift dashboard.
- v2.0: constrained budget optimisation and creator portfolio planning.
- Enterprise track: authentication, object storage, asynchronous batch, registry, and governance.
- Every feature must connect to a user decision and validation plan.

Portfolio defense answer

The next release should deepen trust, not simply add algorithms. I would prioritise per-prediction explanation and persisted outcome monitoring before introducing a more complex model.

Evidence in InfluenceLift AI

- Current roadmap already identifies explainability, MLflow, drift, and optimisation.
- Architecture supports extending the package and API.
- Public v1 is stable and usable.

Trade-off to acknowledge

A roadmap is not a promise. Each stage needs evidence, ownership, and operating cost.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Pitch v1.1 in two minutes to a product manager and a platform engineer.

38. Whiteboard defense sequence

Objective: Practise a structured system-design interview flow.

Core explanation

- Clarify users, scale, data sensitivity, latency, and batch needs.
- State functional and non-functional requirements.
- Draw context, containers, and data flow.
- Deep dive into data quality and model lifecycle.
- Cover API, storage, security, monitoring, scaling, and trade-offs.
- Close with bottlenecks and roadmap.

Portfolio defense answer

I use a layered explanation so the interviewer can redirect depth. I start with the decision and system boundary, then move into the highest-risk domain issue: messy-data reliability. Only after that do I discuss the estimator.

Evidence in InfluenceLift AI

- The system-design diagram pack mirrors this sequence.
- The architecture is small enough to draw in five minutes.
- Each component maps to code in the repository.

Trade-off to acknowledge

Over-explaining implementation before clarifying scale can lead to the wrong architecture.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Complete a 35-minute mock design using the sequence and record your timing.

39. Two-minute project defense

Objective: Deliver a concise, credible project summary.

Core explanation

- Problem: unreliable influencer data and uncertain campaign sales.
- Solution: auditable cleaning plus cross-validated forecasting.
- Product: live predictor, batch flow, simulator, API.
- Engineering: package, tests, CI, Docker, deployment.
- Result: Ridge selected at about 2,000 RMSE and 0.493 R^2 on original case study.
- Limitations: observational, limited features, synthetic public demo.

Portfolio defense answer

InfluenceLift AI is an end-to-end marketing analytics product that cleans inconsistent campaign records and forecasts attributed sales before budget is spent. I compared linear and nonlinear regressors using leakage-safe cross-validation and selected Ridge because it matched or beat the complex challenger while remaining stable and interpretable. I packaged the transformations with the model, exposed single and batch predictions through Streamlit and FastAPI, added empirical uncertainty and extrapolation warnings, tested the system across Python versions, containerised it, and deployed a public demo using synthetic data. The main limitation is that the available features and observational attribution constrain causal and predictive claims.

Evidence in InfluenceLift AI

- Live app and source are publicly accessible.
- Original metrics and demo metrics are labelled separately.
- Manual and automated QA evidence exists in merged PRs.

Trade-off to acknowledge

A strong summary must not imply that the synthetic demo model achieved the original case-study metrics.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Practise the answer at 60, 120, and 300 seconds.

40. Deep-dive defense checklist

Objective: Use a final checklist before interviews or portfolio reviews.

Core explanation

- Know the exact data corruption rules and why they exist.
- Know model comparison metrics and selection rationale.
- Know one limitation and one improvement for every component.
- Be able to draw context, container, sequence, and deployment diagrams.
- Be able to run the demo and recover from common failures.
- Keep role attribution and dataset policy accurate.
- End answers with business impact and evidence.

Portfolio defense answer

The goal is not to memorise every line. It is to connect each design decision to evidence, acknowledge trade-offs, and show how I would evolve the system under different requirements.

Evidence in InfluenceLift AI

- This guide, coding guide, workbook, question bank, and diagram pack form one preparation system.
- The repository provides concrete implementation anchors.
- The live app gives an immediate recruiter proof point.

Trade-off to acknowledge

Over-rehearsed answers can sound rigid. Use the checklist as structure and adapt to the interviewer's priorities.

Likely follow-up: What changes if this moves from a public portfolio demo to a multi-tenant client platform?

Practice prompt: Run a final mock interview with a colleague and score clarity, evidence, trade-offs, and ownership.